TWO WAY DATA BINDING IN ANGULAR

codewithrutu_09

✅What is Two-way Data binding?

Two-way data binding means synchronization between the component and the view.

➡ When the user types in a form field → component updates

➡ When the component value changes → view updates

✅ Best for form controls like inputs, checkboxes, selects, etc.

✨ Makes your UI reactive and reduces manual DOM handling.

codewithrutu_09

Rutuja Shinde

✅ Syntax Overview

Two-way binding uses banana in a box syntax:

app.component.html

```
[(ngModel)]="property"
```

The [ ] is for property binding

The ( ) is for event binding

Combined → [( )]

app.component.html

```
<input [(ngModel)]="username" />
```

codewithrutu_09

Rutuja Shinde

✅ Component Code Example

Component :

app.component.ts

```ts
export class UserComponent {
  username = 'JohnDoe';
}
```

Template :

app.component.html

```html
<p>Hello, {{ username }}!</p>
<input [(ngModel)]="username" />
```

📌 Typing in the input updates username, and username updates the view instantly.

codewithrutu_09 Rutuja Shinde

✅ How It Works

[(ngModel)]="username" is actually a shorthand for:

app.component.html

```
<input [value]="username"
(input)="username = $event.target.value" />
```

✅ It’s both property binding and event binding — done automatically by Angular using FormsModule

🧠 It’s just property binding + event binding bundled together!

codewithrutu_09

Rutuja Shinde

✅ FormsModule is Required!

⚠ Don't forget to import FormsModule to use ngModel:

app.component.ts

```ts
import { FormsModule } from '@angular/forms';

@NgModule({
  imports: [FormsModule]
})
export class AppModule { }
```

❗ If you forget this, Angular will throw a "Can’t bind to ngModel" error.

codewithrutu_09

Rutuja Shinde

✅ Real-Life Use Cases

1. **Login and signup forms**
2. **Search filters**
3. **Reactive display of inputs**
4. **Editing product details in dashboards**

app.component.html

```
<input [(ngModel)]="product.price" />
<p>Price: {{ product.price }}</p>
```

🔸 **Works great in template-driven forms**

🔸 **Quick and easy for small projects or less complex form logic**

codewithrutu_09

Rutuja Shinde

✅ When Not to Use Two-Way Bindinge Use Cases

🔴 Avoid in complex or large-scale forms — prefer Reactive Forms for:

1. Better validation control
2. Testability
3. Scalability

🔸 Use [(ngModel)] for simple or quick forms

🔸 Use FormGroup and FormControl for advanced cases

codewithrutu_09

Rutuja Shinde

codewithrutu_09

Like this post!

Share with your friends

Save it for later

Tell us what you think in comment

codewithrutu_09

Rutuja Shinde